Additional Assignment: Real-time Stock Data (Kafka / SSE)

> **Estimated time:** ~60 minutes
> **Tools allowed:** any AI coding assistant (Cursor AI, VS Code + Copilot, etc.) — vibe coding encouraged!

---

1. Context

During this lab you will consume a **live stream of fictitious stock price ticks** for 50 companies.
The data is produced by a Kafka broker running on the instructor's server and exposed via a simple **HTTP / SSE API** — no Kafka client libraries required.

---

2. Step 1 — Get your API key

Open in the browser:

```
https://add.piotrkojalowicz.dev/
```

1. Enter the **shared class password**: `A@d-$01`
2. Copy the generated **API key** — it is **unique per student**.
   You will need it in every API call.

> ⚠️ The service is open only during class hours (until 19:00).
> Rate limit: first 10 requests free, then max 1 request / 10 s per key.

---

3. Available API endpoints

Base URL: `https://add.piotrkojalowicz.dev`

Authentication

Every request must include the key as a header **or** a query param:

Method	Example
Header	`X-API-Key: <YOUR_KEY>`
Query param	`?api_key=<YOUR_KEY>` (convenient for browser)

`GET /api/tickers`

Returns the full list of 50 companies.

```
curl -H "X-API-Key: <KEY>" "https://add.piotrkojalowicz.dev/api/tickers"
```

Browser:

```
https://add.piotrkojalowicz.dev/api/tickers?api_key=<KEY>
```

`GET /api/latest?ticker=ACME`

Returns the most recent tick for one or more tickers.

```
curl -H "X-API-Key: <KEY>" \
  "https://add.piotrkojalowicz.dev/api/latest?ticker=ACME&ticker=ALFA"
```

Browser:

```
https://add.piotrkojalowicz.dev/api/latest?ticker=ACME&ticker=ALFA&api_key=<KEY>
```

Response format:

```
{
  "data": [
    {
```

```
      "ticker": "ACME",
      "ts": "2026-05-07T15:30:00+02:00",
      "price": 123.45,
      "currency": "PLN",
      "volume": 1200,
      "seq": 42
    }
  ]
}
```

> Data is retained for **~10 minutes** only — older ticks are automatically deleted.

`GET /api/stream?ticker=ACME`

Live stream via **Server-Sent Events (SSE)** — new tick sent every ~second.

```
curl -N -H "X-API-Key: <KEY>" \
  "https://add.piotrkojalowicz.dev/api/stream?ticker=ACME"
```

Browser (live view):

```
https://add.piotrkojalowicz.dev/api/stream?ticker=ACME&api_key=<KEY>
```

Events arrive as:

```
event: tick
data: {"ticker":"ACME","ts":"...","price":124.10,...}
```

---

4. Task — Build 2 small apps

Use any language and framework. AI coding tools are recommended.

App #1: Realtime Dashboard

Goal: show live price updates for selected companies.

Requirements: - User can **select one or more tickers** from the list - App connects to `/api/stream` and shows **live updates** in the UI - Each update shows at minimum: ticker, price, timestamp - Optionally: short price history (last 20–30 ticks) or a mini chart

Hint: Use `EventSource` (JavaScript) or an SSE client library in your language.

App #2: History Downloader / Viewer

Goal: fetch recent data, display it, and save it locally.

Requirements: - User can **select tickers** and a **time range** (e.g. last 1–10 minutes) - App fetches data from `/api/latest` (repeatedly, with a delay ≥10 s) or `/api/stream` - Displays results as a **table or chart** in the UI - Allows **saving to CSV or JSON**

> Remember: data older than ~10 minutes is gone. Design your polling accordingly.

---

5. Deliverables

Create a **new repository** in the ADD GitHub organisation:

- Repository name: `sXXXXX_kafka` (your student ID, e.g. `s12345_kafka`)
- Repository must be public (or accessible to the instructor)

The repository must contain:

- [ ] Source code for **both apps** (each in its own folder)
- [ ] `README.md` for each app with: prerequisites, installation steps, how to configure the API key, how to run
- [ ] `screenshots/` folder (or screenshots embedded in the README) showing working apps with charts/data
- [ ] `.gitignore` — must exclude `.env`, `__pycache__`, `node_modules`, etc.
- [ ] API key must **not** be committed anywhere

---

6. Grading (0–4 points)

#	Criterion	Points
1	Git repo created correctly in ADD org, named `sXXXXX_kafka`	1 pt
2	Full README: prerequisites, install, configure, run	1 pt
3	Screenshots of working apps including charts	1 pt
4	Proper Git usage: ≥3 meaningful commits, `.gitignore`	1 pt

See `ACCEPTANCE.md` for the full checklist.

---

7. List of tickers

Ticker	Company
ACME	ACME Corp.
ALFA	Alfa Technologies
BETA	Beta Retail Group
CASH	CashBank
CLOUD	CloudNine
COAL	Coal Energy
COPR	Copper Mining Co.
DATA	DataWorks
DEVS	DevStudio
DRON	Dronix
ECO	EcoPower
EDU	EduNext
ENRG	Energo
FARM	FarmFoods
FINX	FinX

Ticker	Company
FOOD	FoodBox
FUEL	FuelOne
GAME	GameForge
GRIN	Green Invest
HEAL	HealTech
HOME	HomeBuild
HYPE	Hype Media
INSR	InsureCo
IOT	IoT Systems
JET	Jet Logistics
LABS	Labs Research
LEND	Lendify
LOGI	LogiWare
MALL	Mall Retail
MEDI	MediCare
META	MetaCom
MOBI	MobiTel
MOVE	MoveNow
NET	Netlink
NOVA	Nova Ventures
OILS	OilSands
PARK	Park Realty
PHAR	Pharmax
PLNT	Plantio

Ticker	Company
PROD	Prodigo
QBIT	QBit Quantum
RAIL	Rail Cargo
ROBO	RoboMakers
SAFE	SafeSecurity
SHIP	ShipIt
SHOP	ShopNow
SOLR	Solaris
TEL	TelcoPlus
TRVL	TravelBee
WATR	WaterWorks

---

8. Quick-start example (Python)

```
import requests, os

API_KEY = os.environ["ADD_API_KEY"]   # set in .env or shell
BASE    = "https://add.piotrkojalowicz.dev"
HEADERS = {"X-API-Key": API_KEY}

# list tickers
tickers = requests.get(f"{BASE}/api/tickers", headers=HEADERS).json()
print(tickers["tickers"][:5])

# latest price for ACME
data = requests.get(f"{BASE}/api/latest", params={"ticker": "ACME"},
headers=HEADERS).json()
print(data)
```

SSE stream (Python):

```
import sseclient, requests, os

API_KEY = os.environ["ADD_API_KEY"]
resp = requests.get(
    "https://add.piotrkojalowicz.dev/api/stream",
    params={"ticker": "ACME"},
    headers={"X-API-Key": API_KEY},
    stream=True,
)
for event in sseclient.SSEClient(resp):
    if event.event == "tick":
        print(event.data)
```

JavaScript (browser / Node):

```
const key = "YOUR_KEY";
const es  = new EventSource(
  `https://add.piotrkojalowicz.dev/api/stream?ticker=ACME&api_key=${key}`
);
es.addEventListener("tick", e => console.log(JSON.parse(e.data)));
```

Acceptance criteria & Grading rubric

Grading — 0 to 4 points

#	Criterion	Points
1	**Git repository** — created in the ADD GitHub organisation, named `sXXXXX_kafka` (your student ID)	1 pt
2	**Full usage instructions** — README explains how to build, configure and run both apps; all required elements listed below are present	1 pt
3	**Screenshots of working apps** — at least one screenshot per app showing live data, including charts/tables	1 pt
4	**Proper use of Git** — meaningful commit messages, at least 3–5 commits showing development progress (not one big "final" commit)	1 pt

Total: 4 points

---

1. Git repository (1 pt)

- [ ] Repository created under the **ADD GitHub organisation**
- [ ] Repository name: `sXXXXX_kafka` (replace `XXXXX` with your student ID, e.g. `s12345_kafka`)
- [ ] Repository is **public** or accessible to the instructor
- [ ] API key is **not committed** — use an `.env` file and add it to `.gitignore`

---

2. Required README elements (1 pt)

Each app must have a `README.md` that includes all of the following:

- [ ] **Prerequisites** — what needs to be installed (Python, Node, etc.) and versions

- [ ] **Installation** — step-by-step: clone, install dependencies (`pip install -r requirements.txt` / `npm install` / etc.)
- [ ] **Configuration** — how to set the API key (e.g. `export ADD_API_KEY=...` or `.env` file)
- [ ] **How to run** — exact command to start the app (e.g. `python app.py` / `npm run dev`)
- [ ] **Which endpoints are used** and how

> A good README means someone who has never seen your code can run it in under 5 minutes.

---

3. Screenshots (1 pt)

Must be included in the repository (e.g. `screenshots/` folder or embedded in the README):

- [ ] **App #1** — screenshot showing live price updates for at least 2 tickers, with a chart or history table
- [ ] **App #2** — screenshot showing fetched data for a selected time range, with a chart or table and a downloaded file (CSV/JSON)
- [ ] Screenshots show **real data** (not empty state or mock data)

---

4. Git usage (1 pt)

- [ ] At least **3 commits** with meaningful messages (e.g. `"Add SSE client for live stream"` , `"Add ticker selection UI"`)
- [ ] **No "init" or "final" mega-commits** containing the entire codebase
- [ ] `.gitignore` present and correct (excludes `.env` , `__pycache__` , `node_modules` , etc.)
- [ ] Each app is in its own directory or clearly separated

Functional requirements checklist

App #1 — Realtime Dashboard

- [ ] User can select one or more tickers (from the 50-company list)
- [ ] App connects to `GET /api/stream` (SSE) and shows **live updates**
- [ ] UI displays: ticker symbol, current price, timestamp
- [ ] Chart or scrolling history of last N ticks
- [ ] App handles connection drops gracefully (reconnects or shows an error)

App #2 — History Downloader / Viewer

- [ ] User can select tickers
- [ ] User can specify a time range (last N minutes, up to ~10 min)
- [ ] App fetches data via `GET /api/latest` (polling with ≥10 s interval) or `/api/stream`
- [ ] Results are displayed in the UI (table or chart)
- [ ] Results can be saved as CSV or JSON file

Common (both apps)

- [ ] API key passed via `X-API-Key` header or `api_key` query param
- [ ] API key **not hardcoded** — stored in env variable or `.env` file
- [ ] App runs locally without errors on a clean install